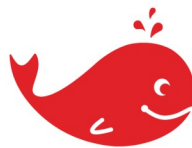


Formation

Introduction au Deep Learning

Séquence 10

Les réseaux autoencodeurs (AE)
Un exemple d'apprentissage non supervisé



FIDLE



Cette session va être enregistrée.
Retrouvez-nous sur notre chaine YouTube :-)

This session will be recorded.
Find us on our YouTube channel :-)

<https://fidle.cnrs.fr/youtube>

Formation

Introduction au Deep Learning

Les réseaux autoencodeurs (AE)
Un exemple d'apprentissage non supervisé



<https://fidle.cnrs.fr>

Powered by CNRS CRIC, and UGA DGDSI
of Grenoble, Thanks !



Course materials (pdf)



Practical work environment*



Corrected notebooks



Videos (YouTube)

You can also subscribe to :



<http://fidle.cnrs.fr/listeinfo>
Fidle information list



<https://listes.services.cnrs.fr/www/info/devlog>
List of ESR* « Software developers » group



<https://listes.math.cnrs.fr/www/info/calcul>
List of ESR* « Calcul » group

(*) ESR is Enseignement Supérieur et Recherche, french universities and public academic research organizations

Some good addresses

Environnements :

TensorFlow :

Keras :

PyTorch :

<https://www.tensorflow.org>

<https://keras.io>

<https://pytorch.org>

Computing resources

IDRIS :

GENCI :

<https://www.idris.fr>

<https://www.genci.fr>

References :

Paper with code :

<https://paperswithcode.com>

Datasets and examples :

Kaggle :

Keras examples :

<https://www.kaggle.com>

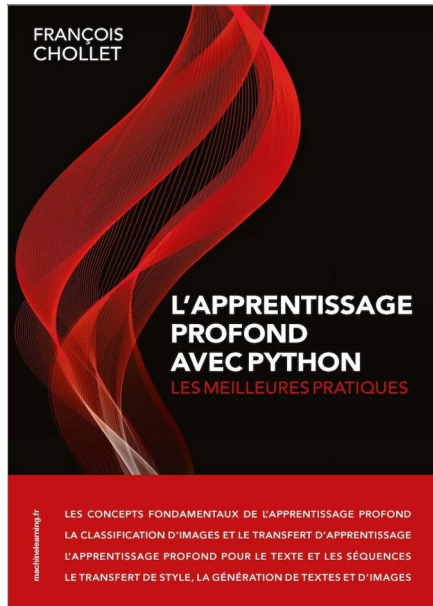
<https://keras.io>



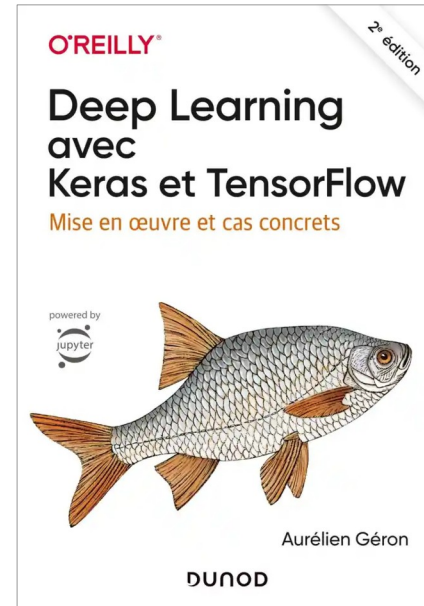
<https://stackoverflow.com>

Some good addresses

Disponible dans 54 bibliothèques universitaires.



L'apprentissage profond
avec Python
François Chollet
Machinelearning.fr (Juillet 2020)
ISBN : 978-2-491674-00-7



Deep Learning avec Keras
et TensorFlow - 2e édition
Aurélien Géron
Dunod (Mars 2020)
ISBN : 978-2-10-079066-1



1 History, Fundamental Concepts	2 Hight Dimensionnall Data CNN	3	4 Demystify mathematics for neural networks.	5 Training strategies Evaluation Sparse data (text) Embedding	6 Sequences data RNN
7 Basic Regression DNN Basic Classification DNN	8 PyTorch A small detour with PyTorch.	9 «Attention is All You Need» Transformers	10 Graph Neural Network GNN	11 Variational Antoencoder VAE	12 Project session «My project in 180 s»
13 Generative Adversarial Networks GAN	14 Diffusion Model Text to image	15 AI, Law, Society and Ethics	16 Model and training optimization Resource efficiency	17 Jean-Zay GPU acceleration	18 Physics-Informed Neural Networks PINNS
19 Deep Reinforcement Learning RL	20 What will be tomorrow's AI Review & perspectives !	21	22	23	24

20 Séquences
du 17 novembre
au 14 mai 2023



SAISON
22/23

10



**Autoencoder
networks**

AE

8.1

Autoencoder network

→ Concepts and architecture

8.2

Keras

→ Functional API

8.3

Example 1 : AE1/3

→ Simple denoiser model



8.4

Example 2 : AE4

→ Dual output model : denoiser and classifier



8.5

Example 3 : AE5

→ Dual output model : denoiser and classifier
with inception approach



10



**Autoencoder
networks**

AE

8.1

Autoencoder network

→ Concepts and architecture

8.2

Keras

→ Functional API

8.3

Example 1 : AE1/3

→ Simple denoiser model



8.4

Example 2 : AE4

→ Dual output model : denoiser and classifier



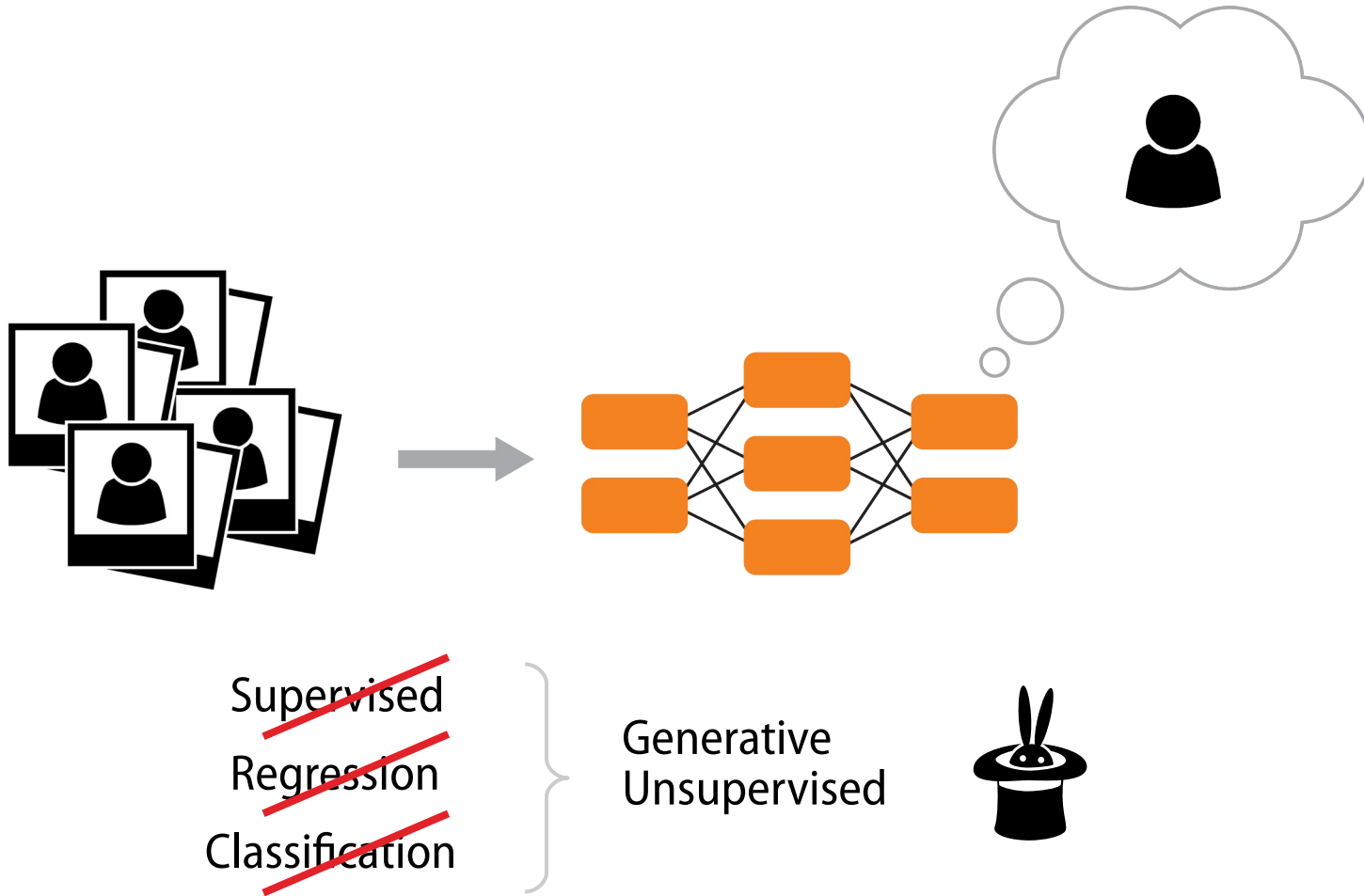
8.5

Example 3 : AE5

→ Dual output model : denoiser and classifier
with inception approach

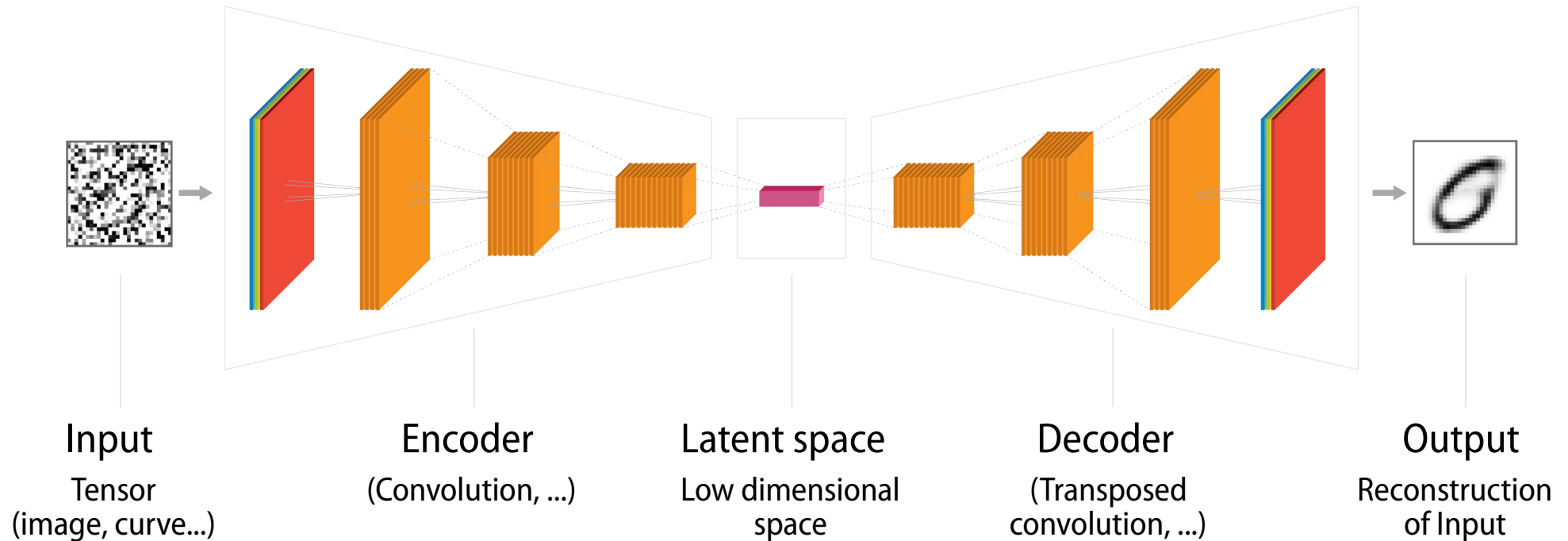


Concepts & architecture



Concepts & architecture

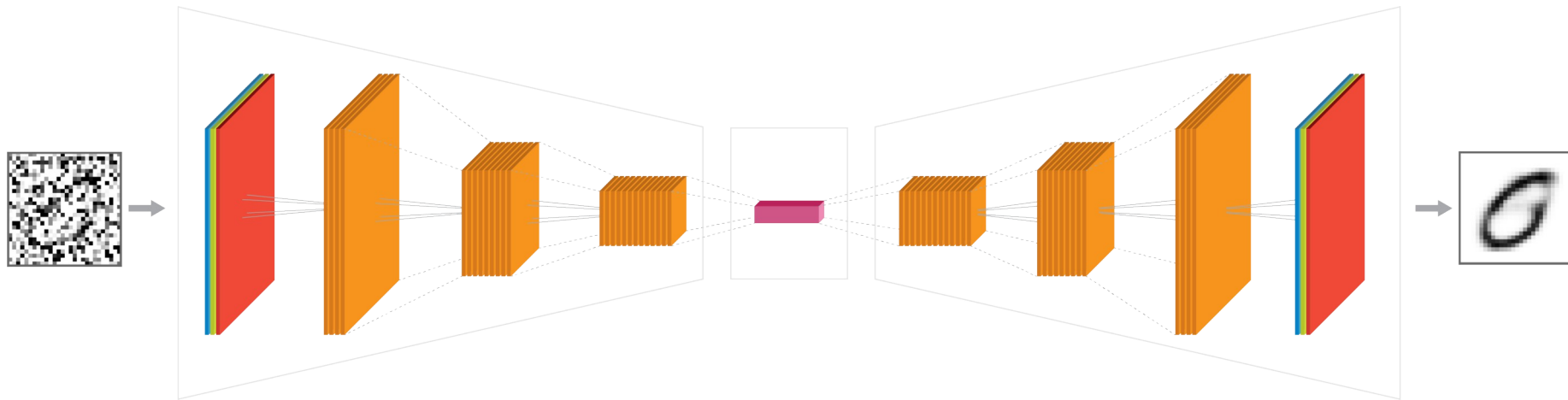
Autoencoders are trained to minimise reconstruction errors.



Note : Encoder/Decoder can use convolutional layers or anything else ;-)

Autoencoder

Autoencoders are trained to minimise reconstruction errors.

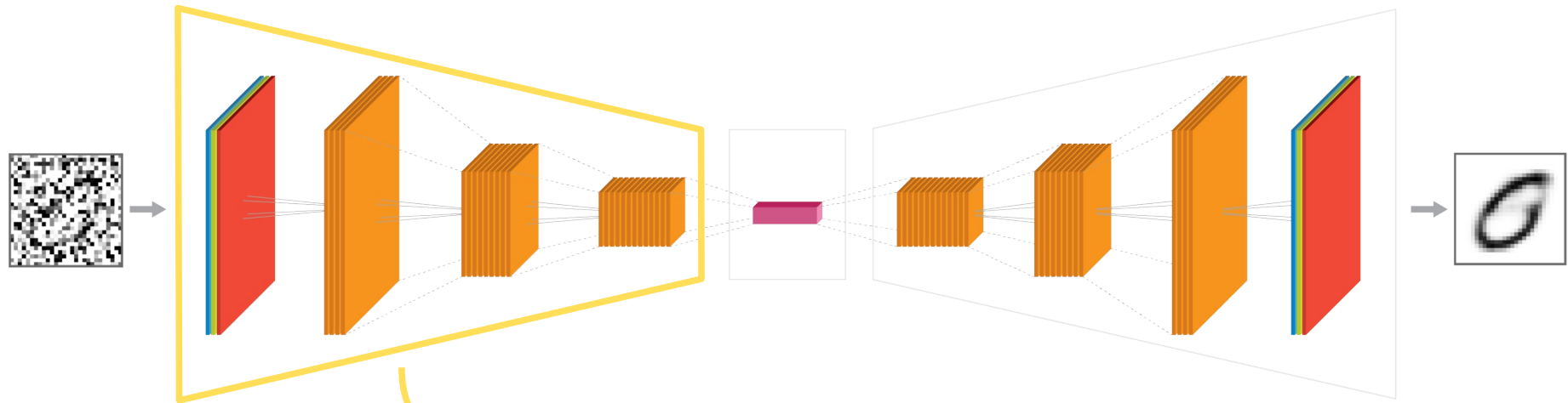


Great, but...

How to build an autoencoder?

Autoencoder

An autoencoders network have 2 parts :

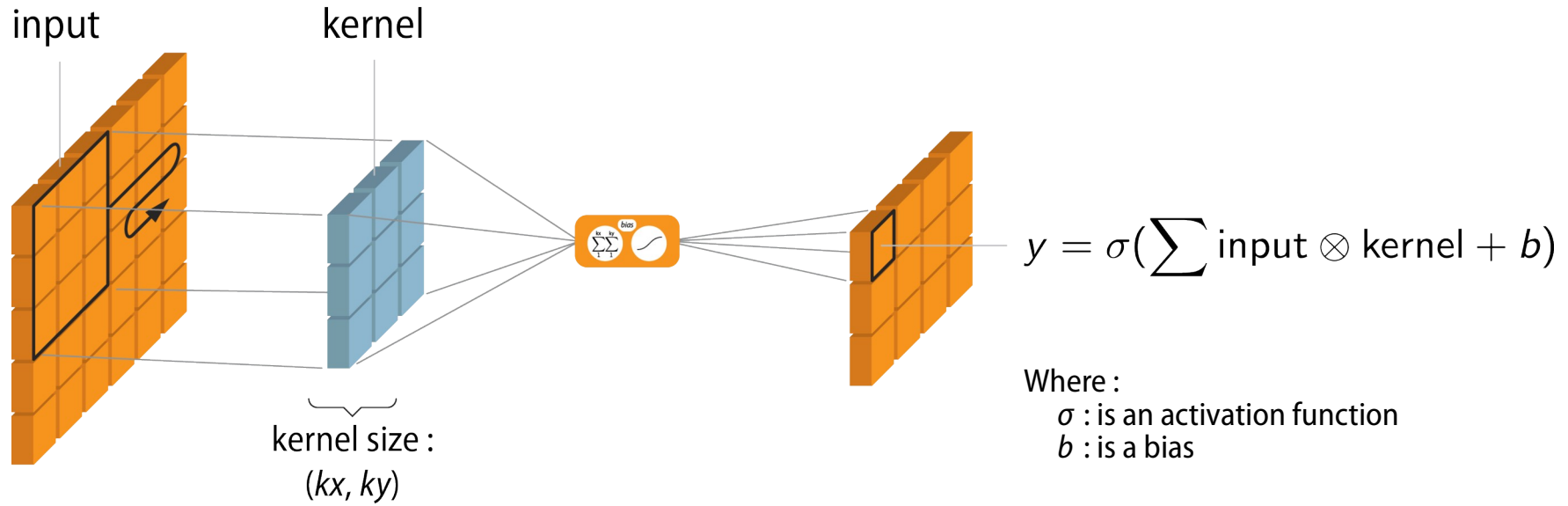


Encoder

Which can use, for example,
some convolutional layers

Convolutional layers

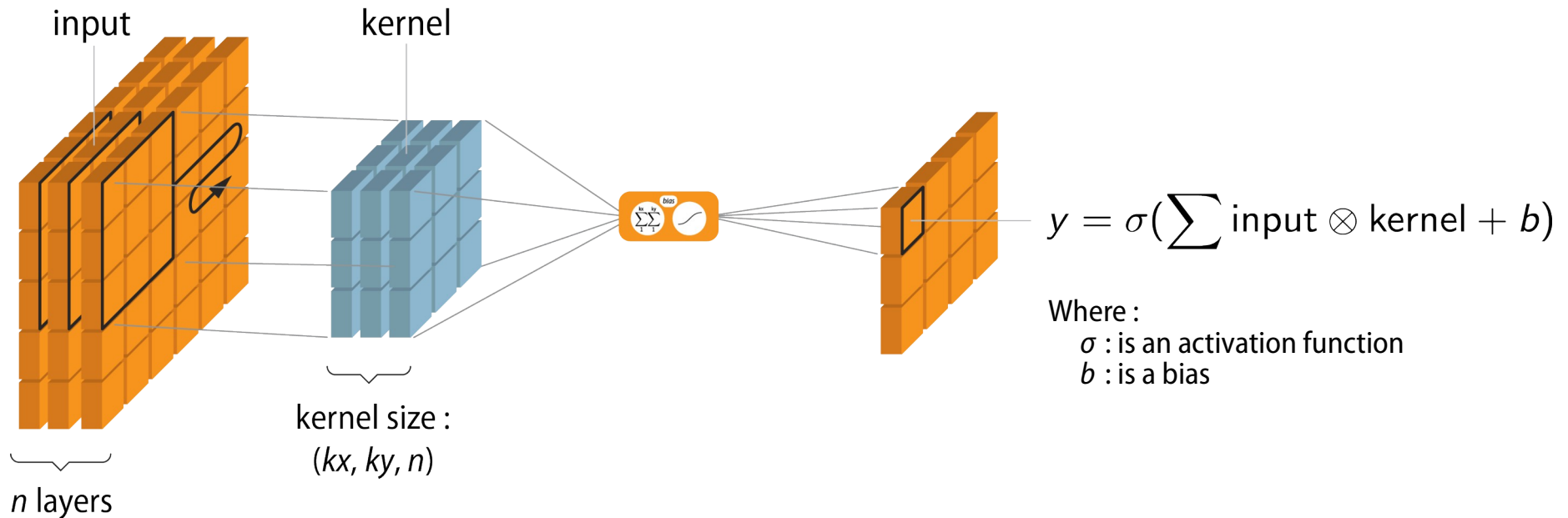
Objective is to **downsampling** an input :



Number of parameters for a convolutional layer : $kx \cdot ky + 1$

Convolutional layers

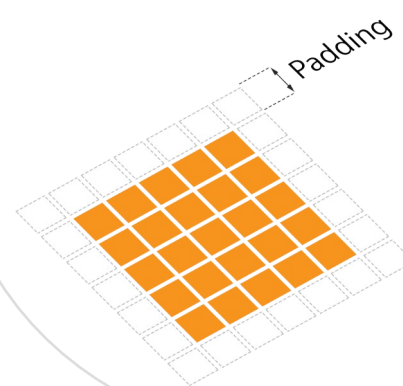
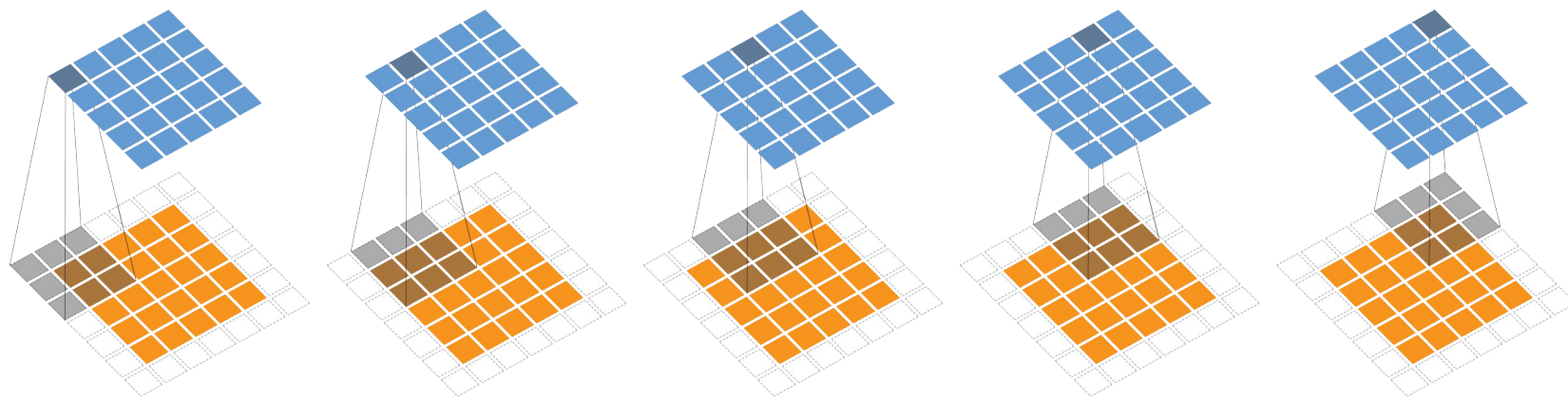
Objective is to **downsampling** an input :



If we want to generate m convolutional layers, we will need m convolutional neurons

Convolution parameters

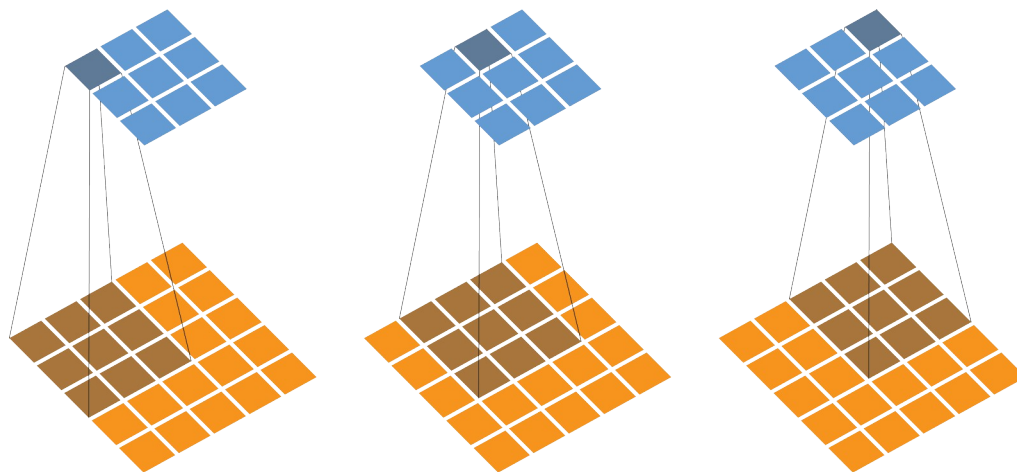
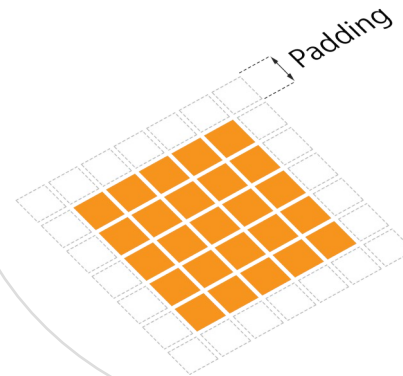
Parameters of a convolutional layer : padding



Keras : padding = 'same' $\Rightarrow$ size is preserved

Convolution parameters

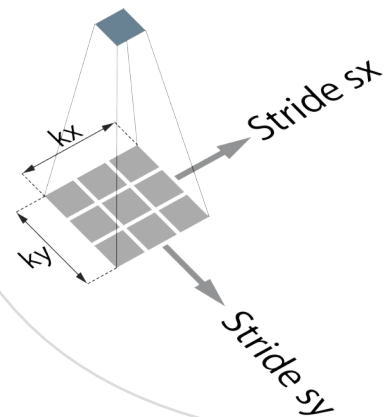
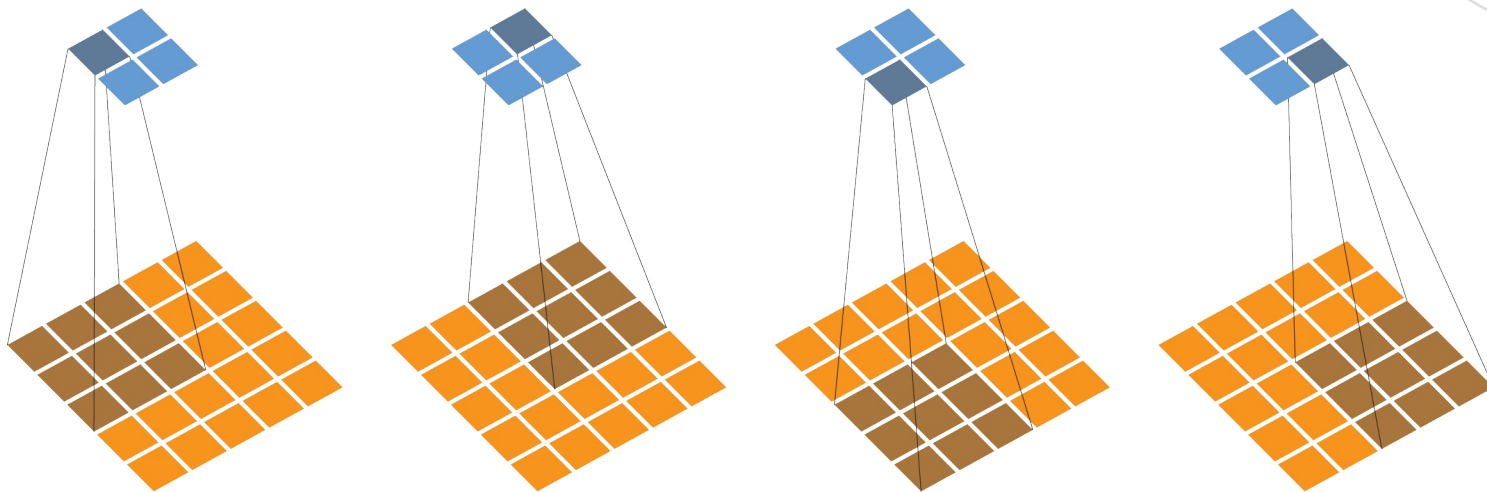
Parameters of a convolutional layer : padding



Keras : padding = 'valid' $\Rightarrow$ Size is not preserved (no padding)

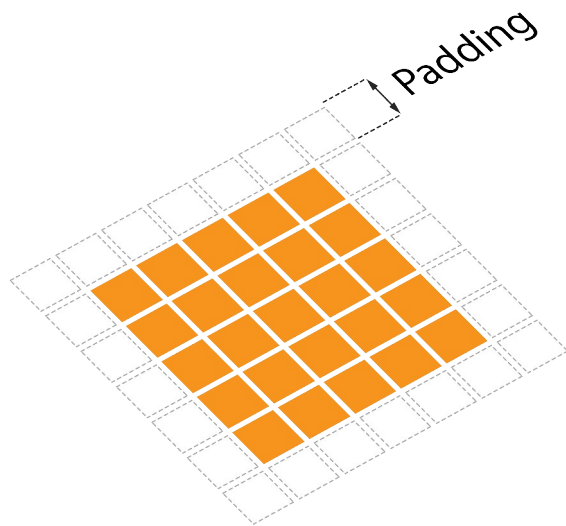
Convolution parameters

Parameters of a convolutional layer : **Strides**



$\text{strides} = (s_x, s_y) \Rightarrow$ A $\text{strides}=(2,2)$ will reduce by 2 the output size

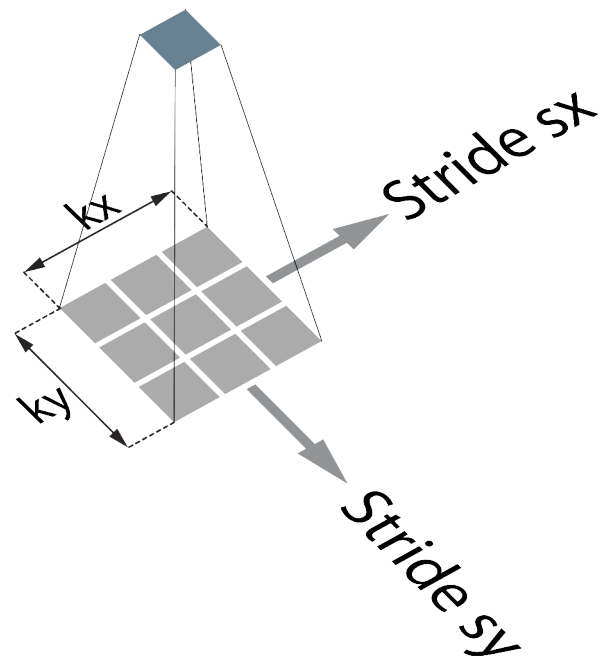
Convolution parameters



padding

'same'

'valid'

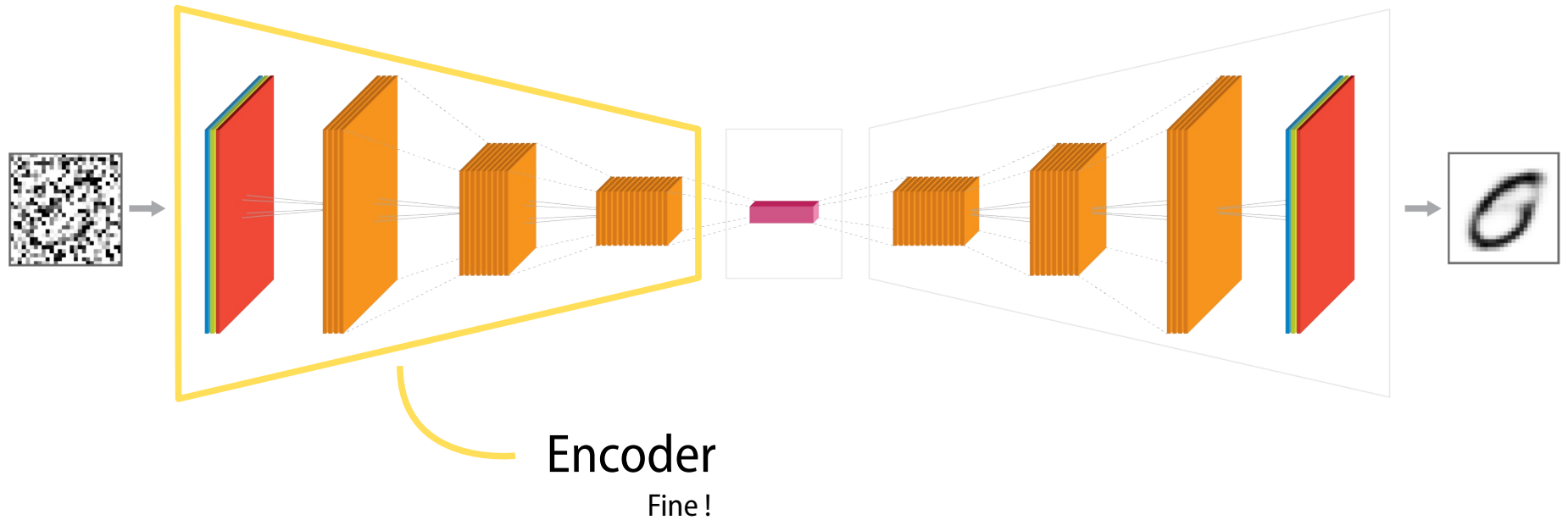


kernel size

strides

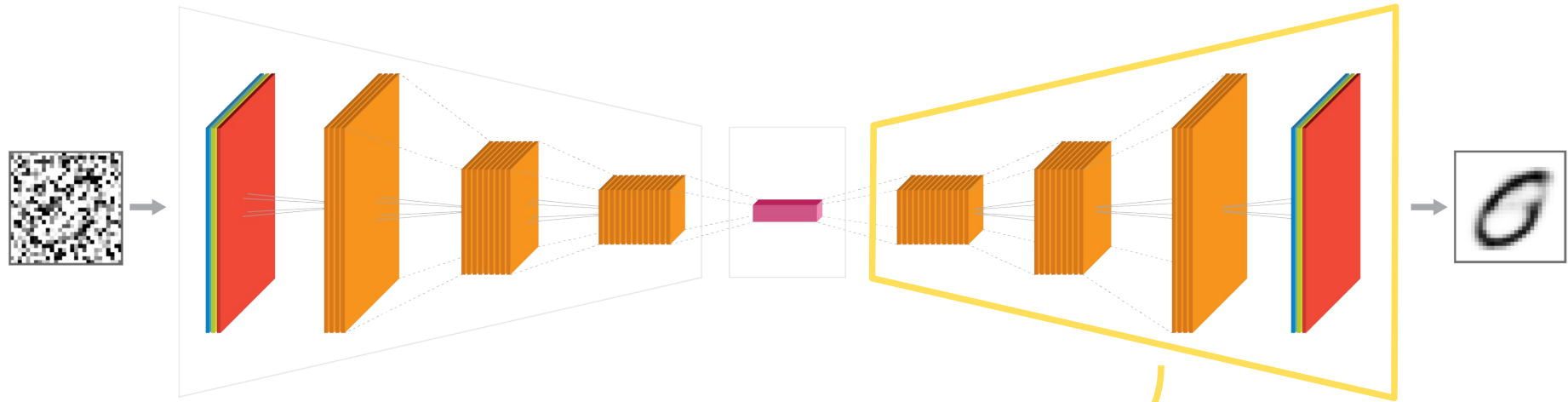
Autoencoder

An autoencoders network have 2 parts :



Autoencoder

An autoencoders network have 2 parts :

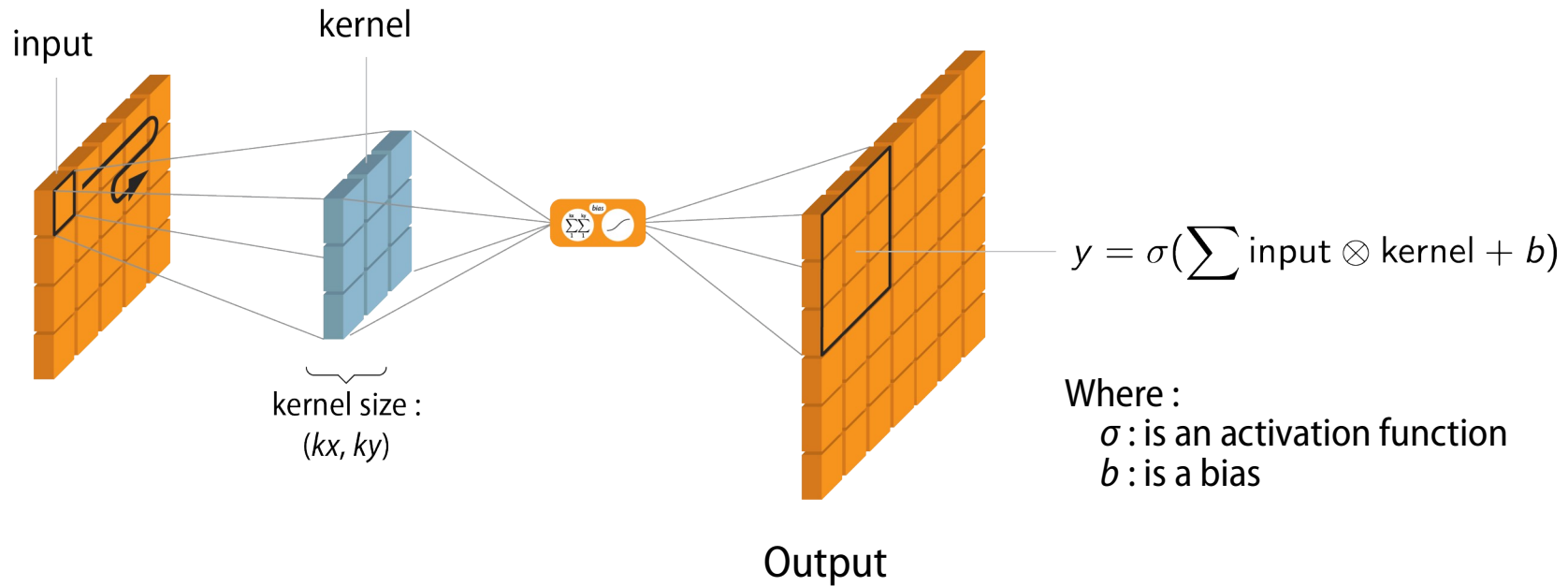


Decoder

Which can use, for example,
transposed convolutional layers

Transposed convolution

Objective is to **upsampling** an input :



Transposed convolution

Basic example :

Input image : (2,2)

Kernel : (2,2)

Output image : (3,3)

1	3
0	2

 $\otimes$

1	3
0	2

 = $\sum$

1	3	0
0	2	0
0	0	0

0	3	9
0	0	6
0	0	0

0	0	0
0	0	0
0	0	0

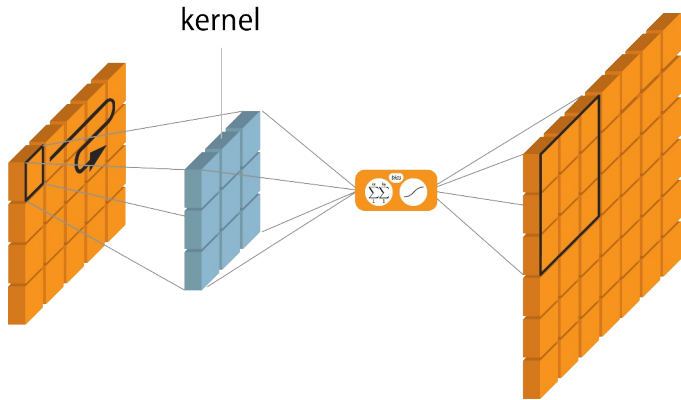
0	0	0
0	2	6
0	0	4

=

1	6	9
0	4	12
0	0	4

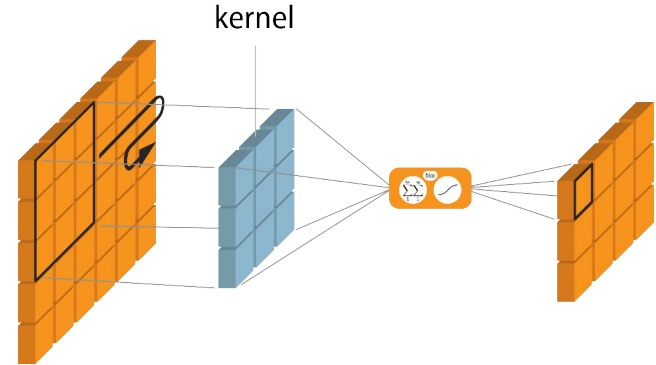
Transposed convolution

! ..But with a very different meaning for the parameters padding and strides !



Transposed convolution

Can be
considered as
equivalent to :



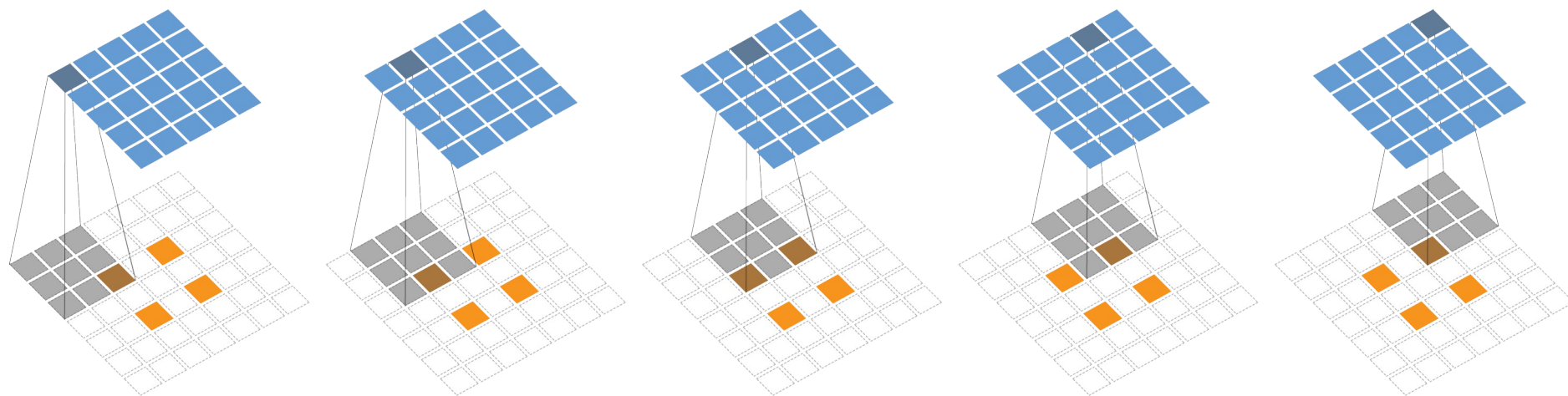
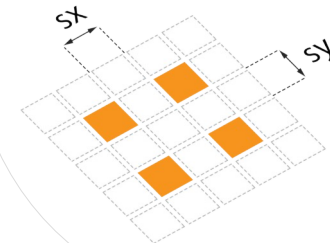
Classical convolution

See : Vincent Dumoulin, Francesco Visin - A guide to convolution arithmetic for deep learning
<https://arxiv.org/abs/1603.07285>
https://github.com/vdumoulin/conv_arithmetic (illustrations)

Transposed convolution

Parameters of a
transposed convolutional layer :

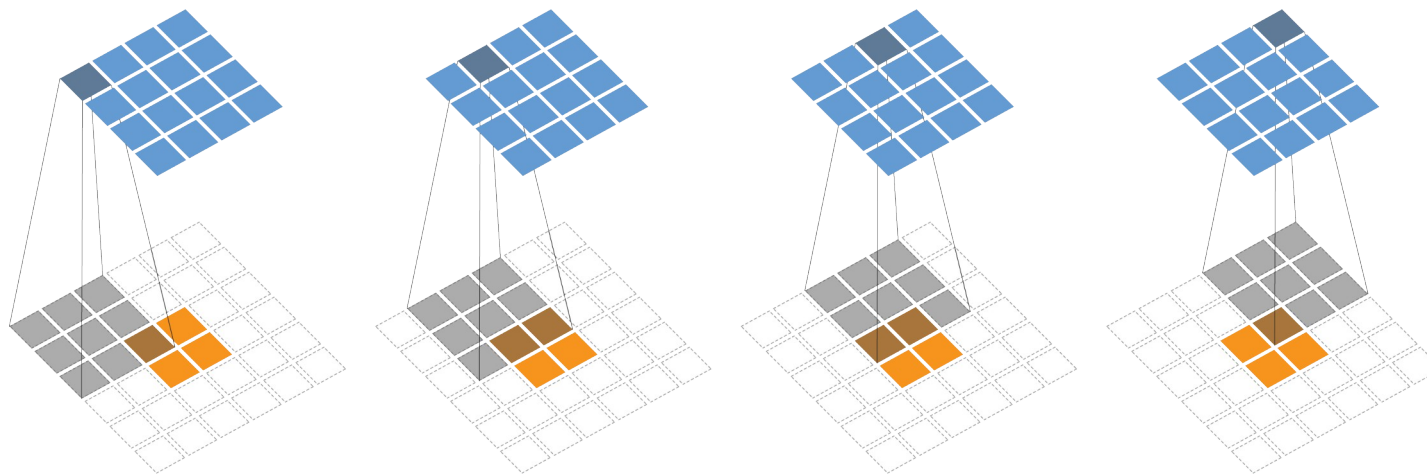
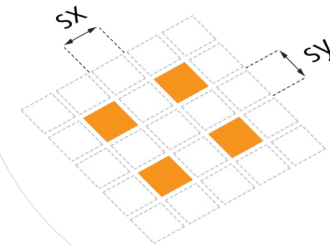
strides, padding



Transposed convolution, with no padding and strides=(2,2)

Transposed convolution

Parameters of a
transposed convolutional layer : **strides, padding**

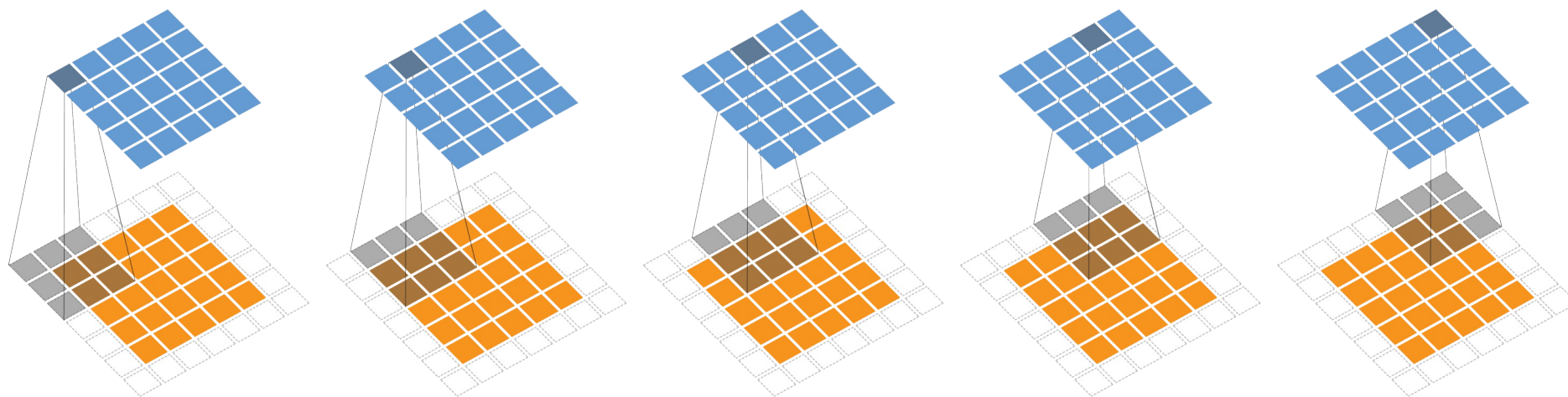
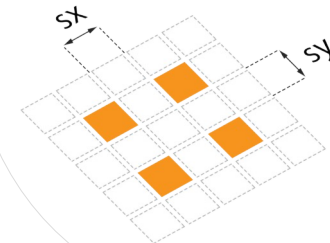


Transposed convolution, with **no padding** and **strides=(1,1)**

Transposed convolution

Parameters of a
transposed convolutional layer :

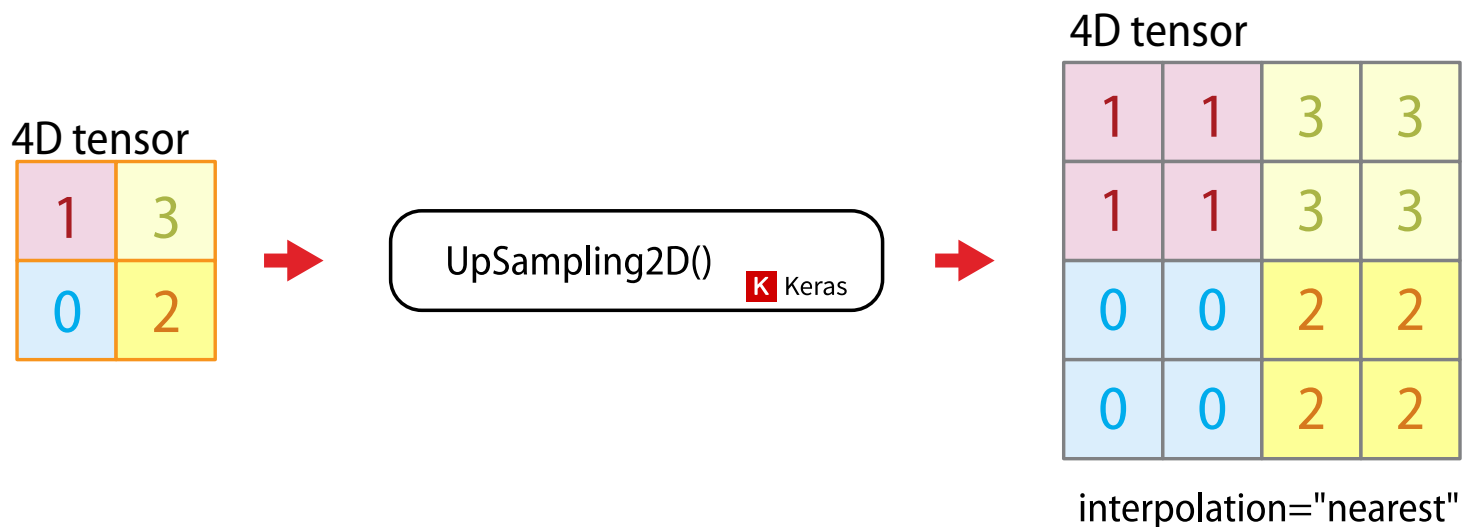
strides, padding



Transposed convolution, with padding='same' and strides=(1,1)

Upsampling

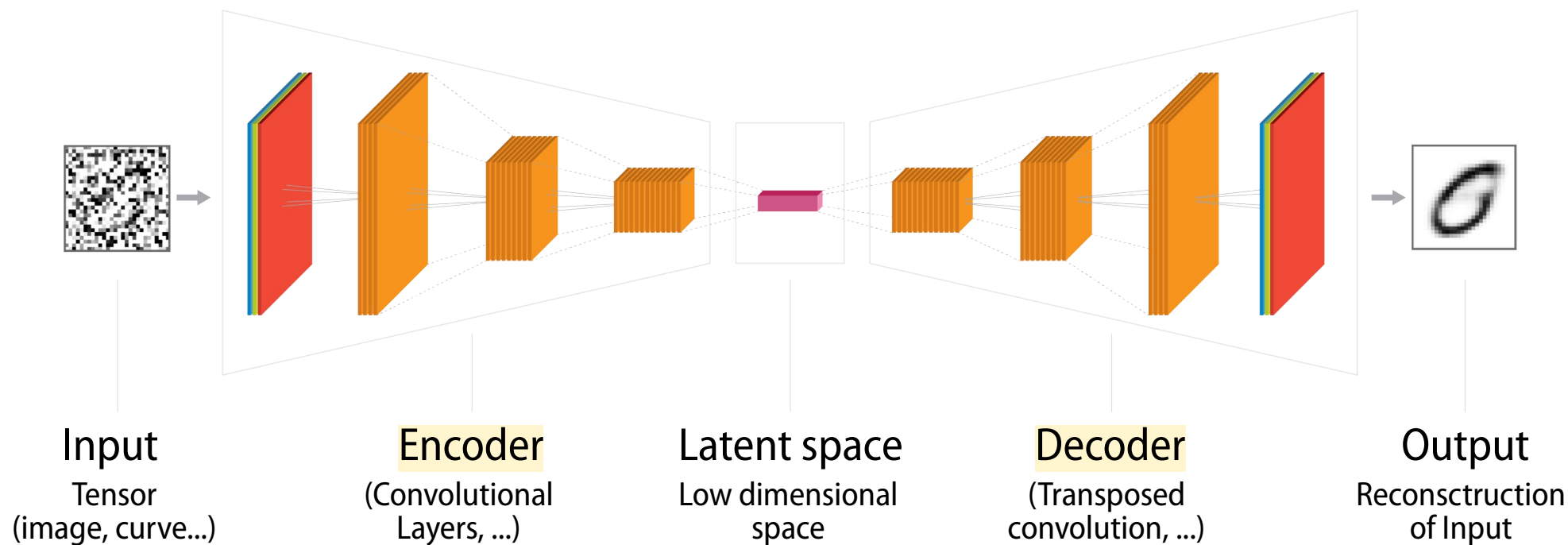
Objective is to ...upsampling an input :



Very easy to use and limits artifacts
Typically used by adding a convolution layer after

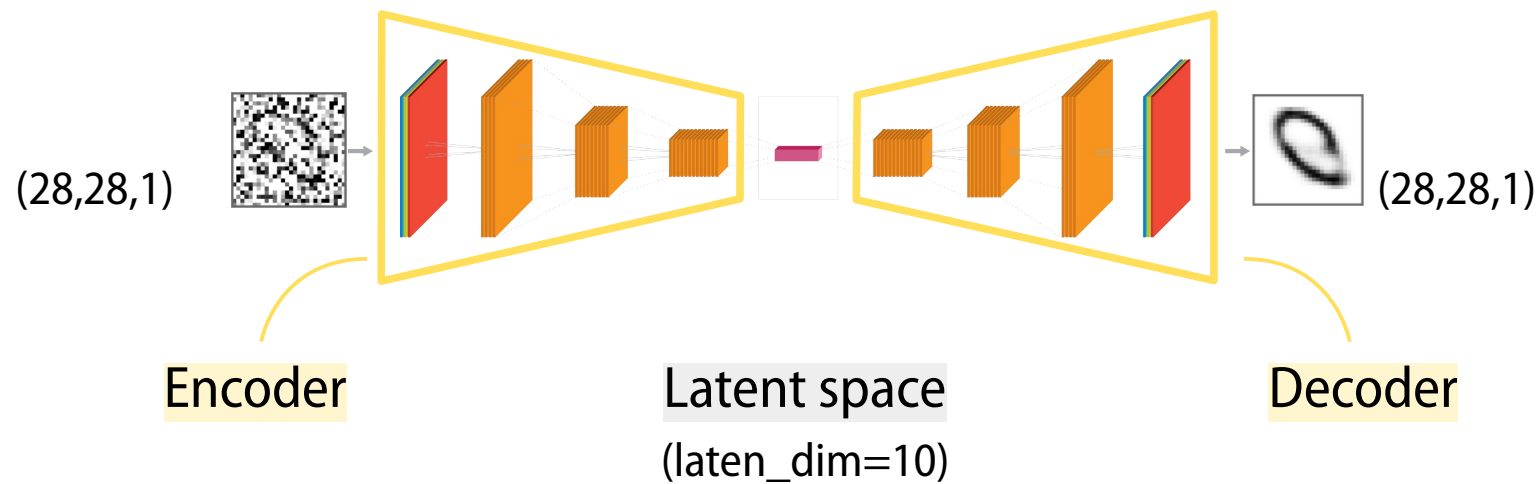
(Could be "bilinear")

Autoencoder



Note : Encoder/Decoder can use convolutional layers or anything else ;-)

Autoencoder



Layer (type)	Output Shape
input_1 (InputLayer)	[(None, 28, 28, 1)]
conv2d (Conv2D)	(None, 14, 14, 32)
conv2d_1 (Conv2D)	(None, 7, 7, 64)
flatten (Flatten)	(None, 3136)
dense (Dense)	(None, 16)
dense_1 (Dense)	(None, 10)

Layer (type)	Output Shape
input_2 (InputLayer)	[(None, 10)]
dense_2 (Dense)	(None, 3136)
reshape (Reshape)	(None, 7, 7, 64)
conv2d_t1 (Conv2DT)	(None, 14, 14, 64)
conv2d_t2 (Conv2DT)	(None, 28, 28, 32)
conv2d_t3 (Conv2DT)	(None, 28, 28, 1)

10



**Autoencoder
networks**

AE

8.1

Autoencoder network

→ Concepts and architecture

8.2

Keras

→ Functional API

8.3

Example 1 : AE1/3

→ Simple denoiser model



8.4

Example 2 : AE4

→ Dual output model : denoiser and classifier



8.5

Example 3 : AE5

→ Dual output model : denoiser and classifier
with inception approach



About Keras...



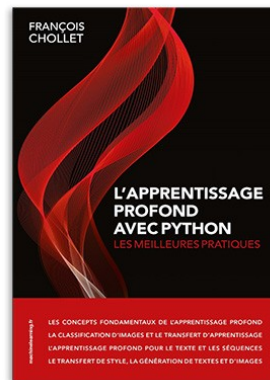
« Deep learning for humans. »

Keras is a full part of **TensorFlow** !
By installing Tensorflow, we install Keras :-)

<https://keras.io/>

<https://www.tensorflow.org/guide/keras>

https://www.tensorflow.org/api_docs/python/tf/keras



By François Chollet
(Traduction en français)
ISBN 978-2-491674-00-7.

Sequential API



Extremely simple API :-)
Perfectly adapted to
simple things !

```
model = keras.Sequential()  
model.add( keras.Input(shape=(28,28)) )  
model.add( keras.layers.Flatten() )  
model.add( keras.layers.Dense( hidden1, activation='relu' ) )  
model.add( keras.layers.Dense( hidden2, activation='relu' ) )  
model.add( keras.layers.Dense( 10,          activation='softmax' ) )
```

Simple is beautiful !



But how to implement a model
with several inputs or outputs ?

Functional API

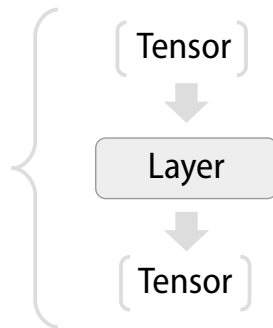
```
inputs = keras.Input(shape=(28,28))

x      = keras.layers.Flatten()(inputs)
x      = keras.layers.Dense( hidden1, activation='relu')(x)
x      = keras.layers.Dense( hidden2, activation='relu')(x)
outputs = keras.layers.Dense( 10,      activation='softmax')(x)

model = keras.Model(inputs=inputs, outputs=outputs)
```



Each layer is
a function:

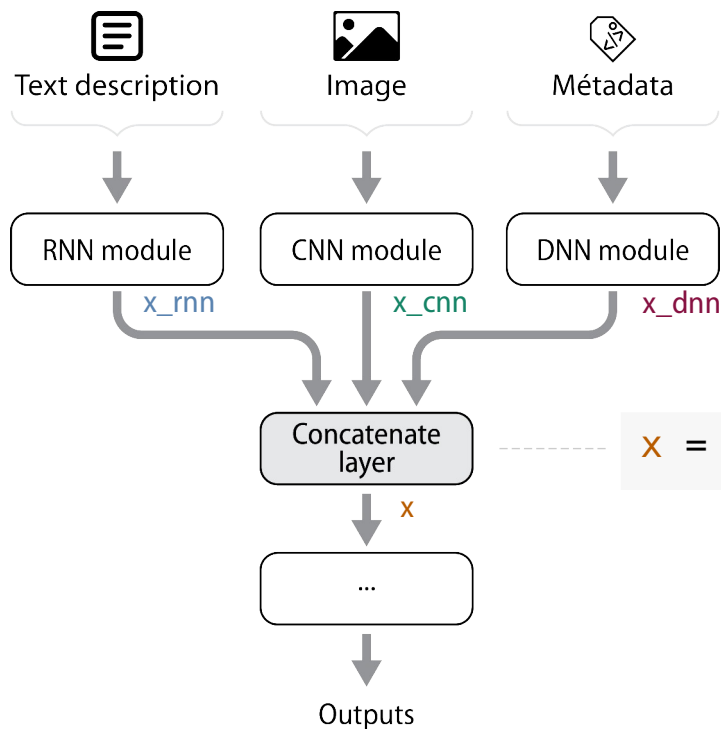


The model is created by specifying
the extremities of a graph
constituted by the layers chain.

As simple as beautiful !



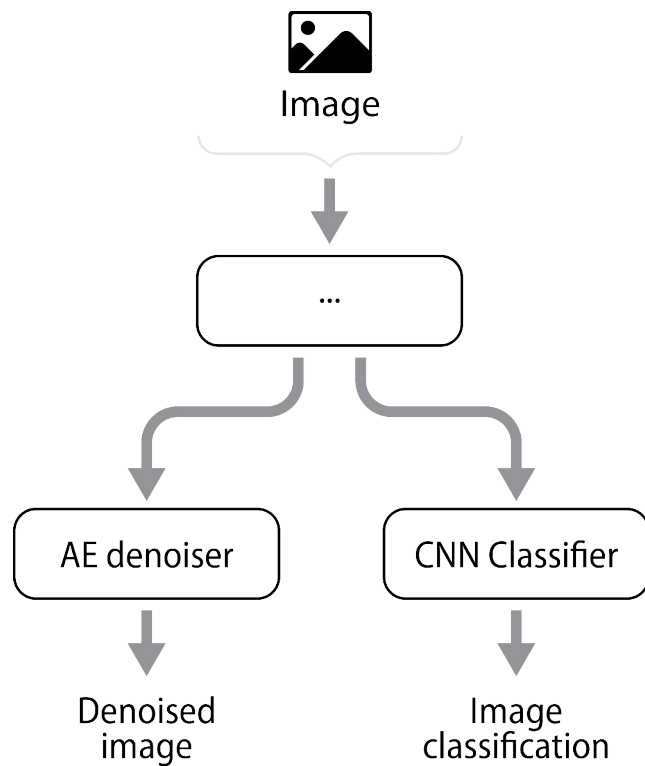
Multiple input model



It is very easy to have different inputs with specific training and and concatenate the result for common training.

```
x = keras.concatenate( [x_rnn, x_cnn, x_dnn], axis=-1)
```

Multiple output model



```
inputs      = keras.Input(shape=(28, 28, 1))  
  
denoised     = ae(inputs)  
classcat     = cnn(inputs)  
  
model = keras.Model(inputs, [denoised, classcat])
```

[AE4] Notebook

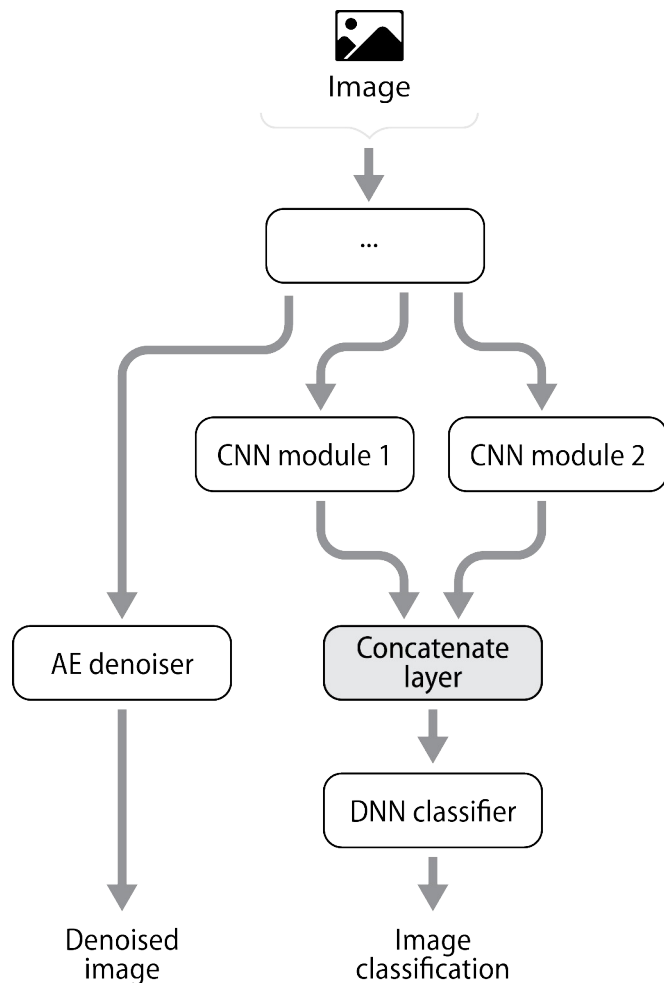


A model can be used as a simple layer

It is therefore very easy to design a model in a modular way...

Like with Legos !

Inception model



```
inputs = keras.Input(shape=(28, 28, 1))  
  
denoised = ae(inputs)  
  
x_cnn1 = cnn1(inputs)  
x_cnn2 = cnn2(inputs)  
  
x = keras.layers.concatenate([x_cnn1,x_cnn2], axis=1)
```

[AE5] Notebook



The idea is to be able to do different training in parallel.

Each branch learning specifically certain characteristics.

Compilation and fit



Compilation : In case of multiple outputs, we will specify a loss function for each output :

```
model.compile( optimizer = 'adam',  
               loss       = [ categorical_crossentropy, mse ]  
               loss_weights = [ 0.25, 1. ] )
```



Fit : In case of multiple inputs (outputs), an input (outputs) list will be given, for example :

```
model.fit( [ x_train1, x_train2 ], [ y_train1, y_train2 ],  
           epochs = 10, batch_size = 64 )
```

10



**Autoencoder
networks**

AE

8.1

Autoencoder network

→ Concepts and architecture

8.2

Keras

→ Functional API

8.3

Example 1 : AE1/3

→ Simple denoiser model



8.4

Example 2 : AE4

→ Dual output model : denoiser and classifier



8.5

Example 3 : AE5

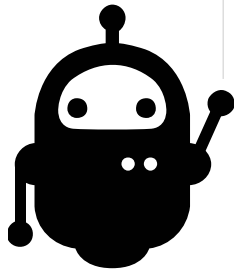
→ Dual output model : denoiser and classifier
with inception approach





Simple denoiser model

Notebook : [\[AE1/2\]](#)



Objectives :

Denoising heavily noisy images

Implement a **denoiser AE**

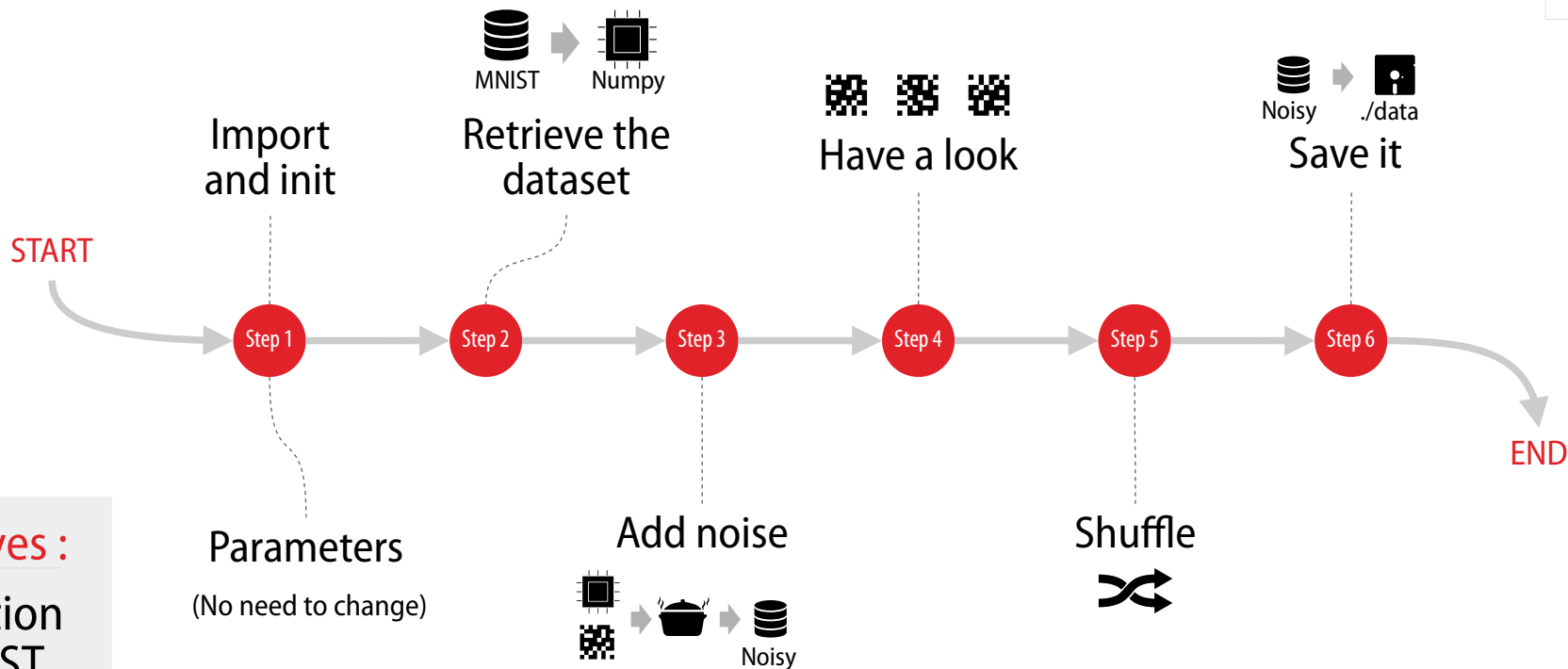
Understand the Keras **functional API**

Dataset :

MNIST



50 s !



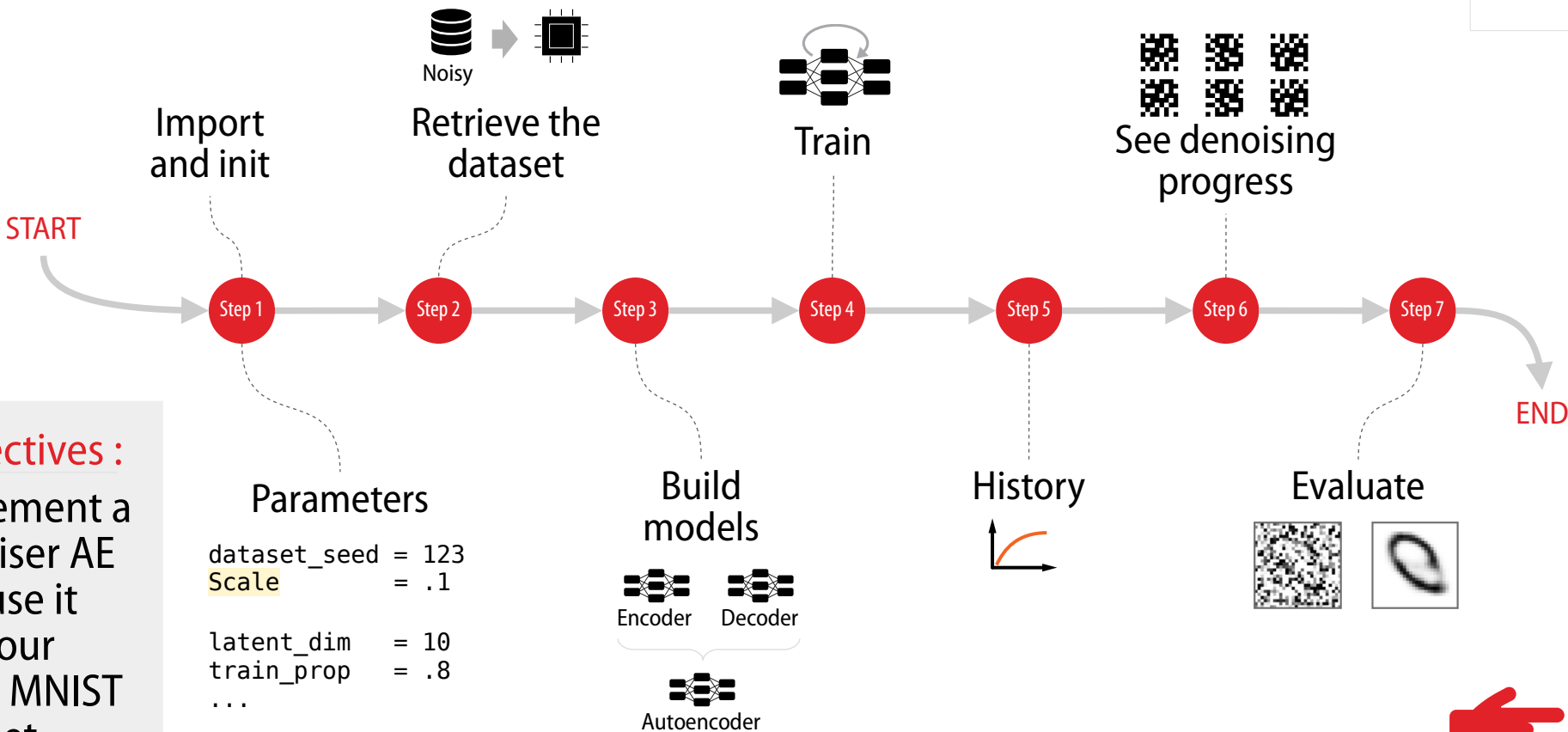
Objectives :

Preparation
of a MNIST
dataset
containing
noisy images





20' on a CPU
With scale=1



Objectives :

Implement a denoiser AE and use it with our noisy MNIST dataset



10



**Autoencoder
networks**

AE

8.1

Autoencoder network

→ Concepts and architecture

8.2

Keras

→ Functional API

8.3

Example 1 : AE1/3

→ Simple denoiser model



8.4

Example 2 : AE4

→ Dual output model : denoiser and classifier



8.5

Example 3 : AE5

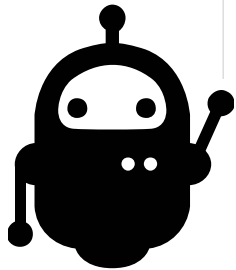
→ Dual output model : denoiser and classifier
with inception approach





Dual output model

Notebook : [\[AE4\]](#)



Objective :

Using an uniq model for denoising and classifying

Implement a dual output model

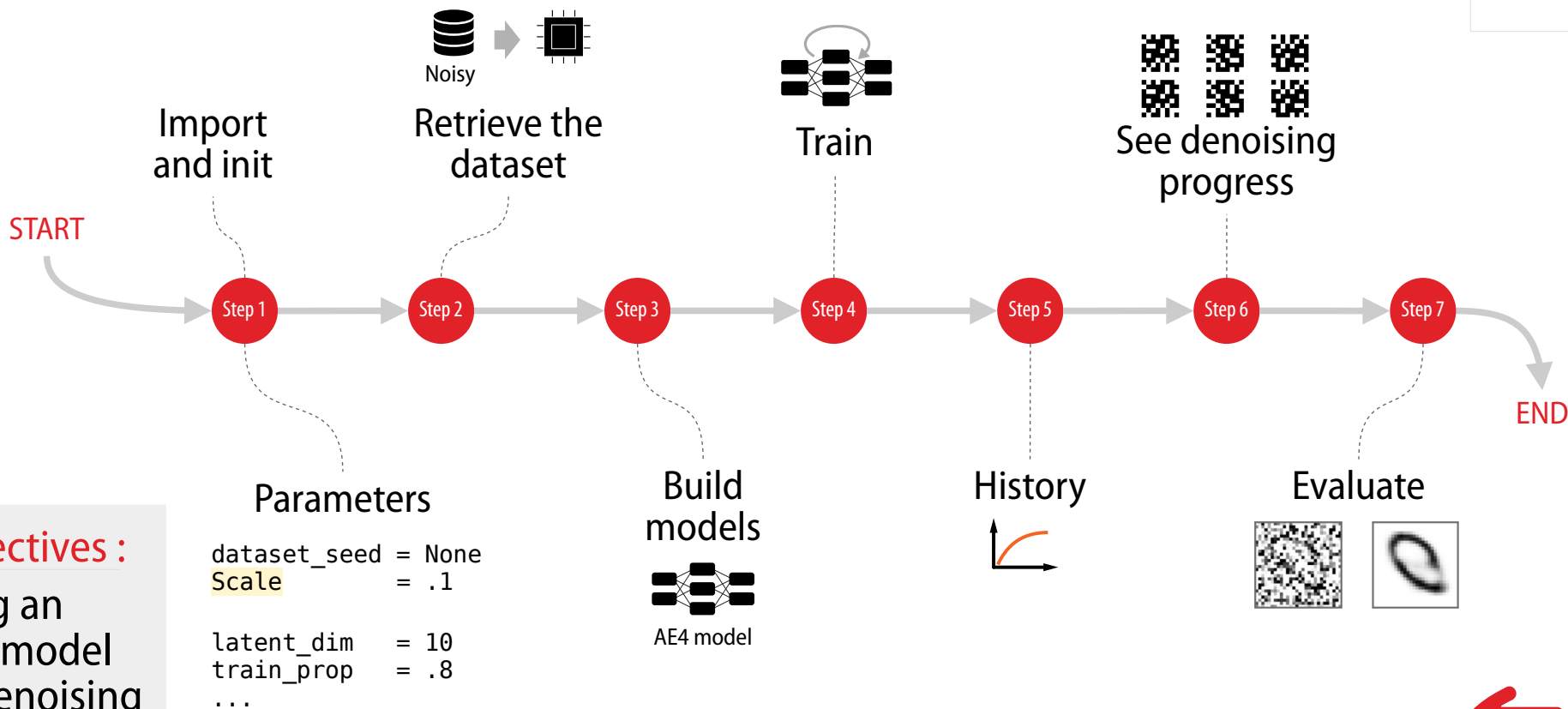
Understand the Keras functional API

Dataset :

MNIST



20' on a CPU
With scale=1



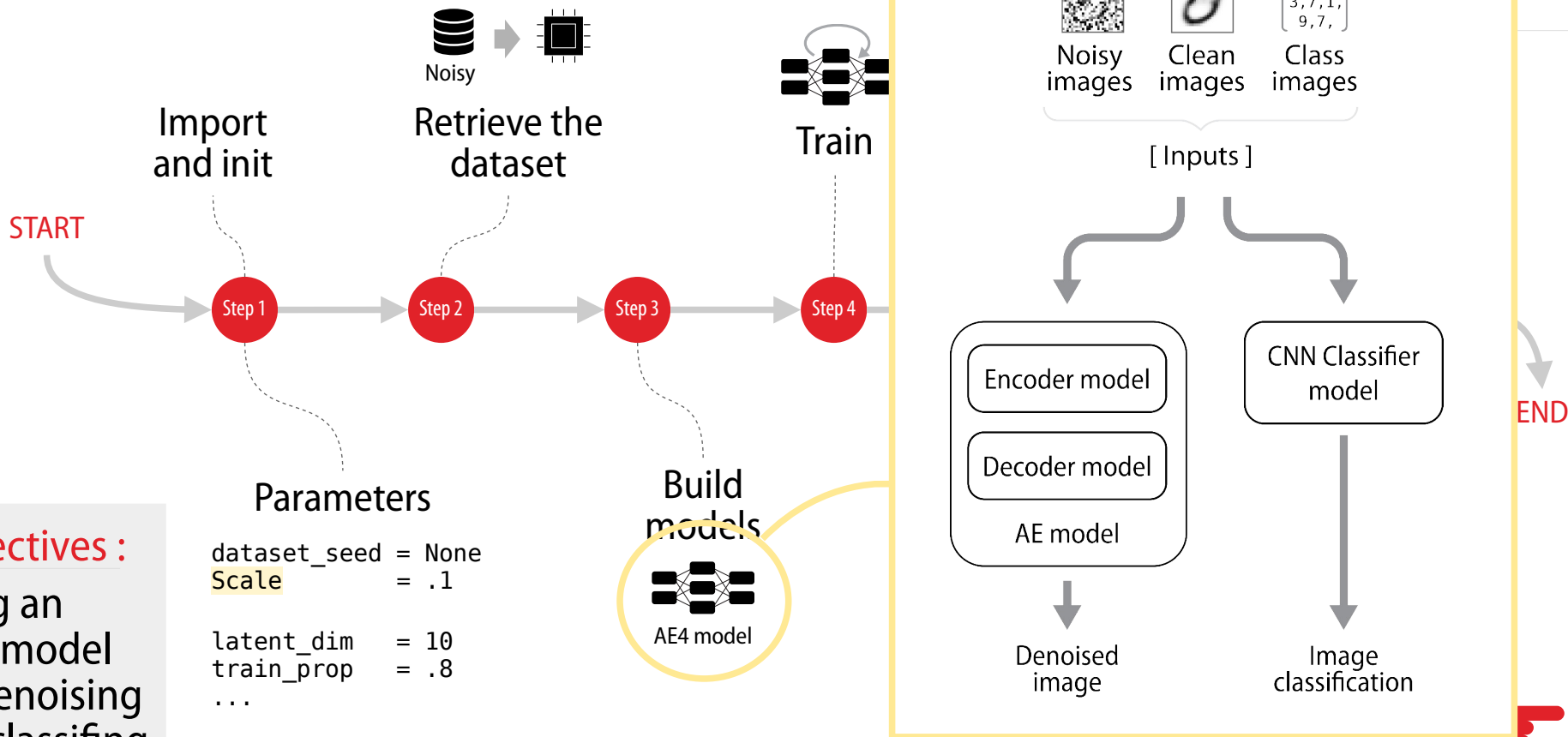
Objectives :

Using an
uniq model
for denoising
and classifing





0' on a CPU
with scale=1

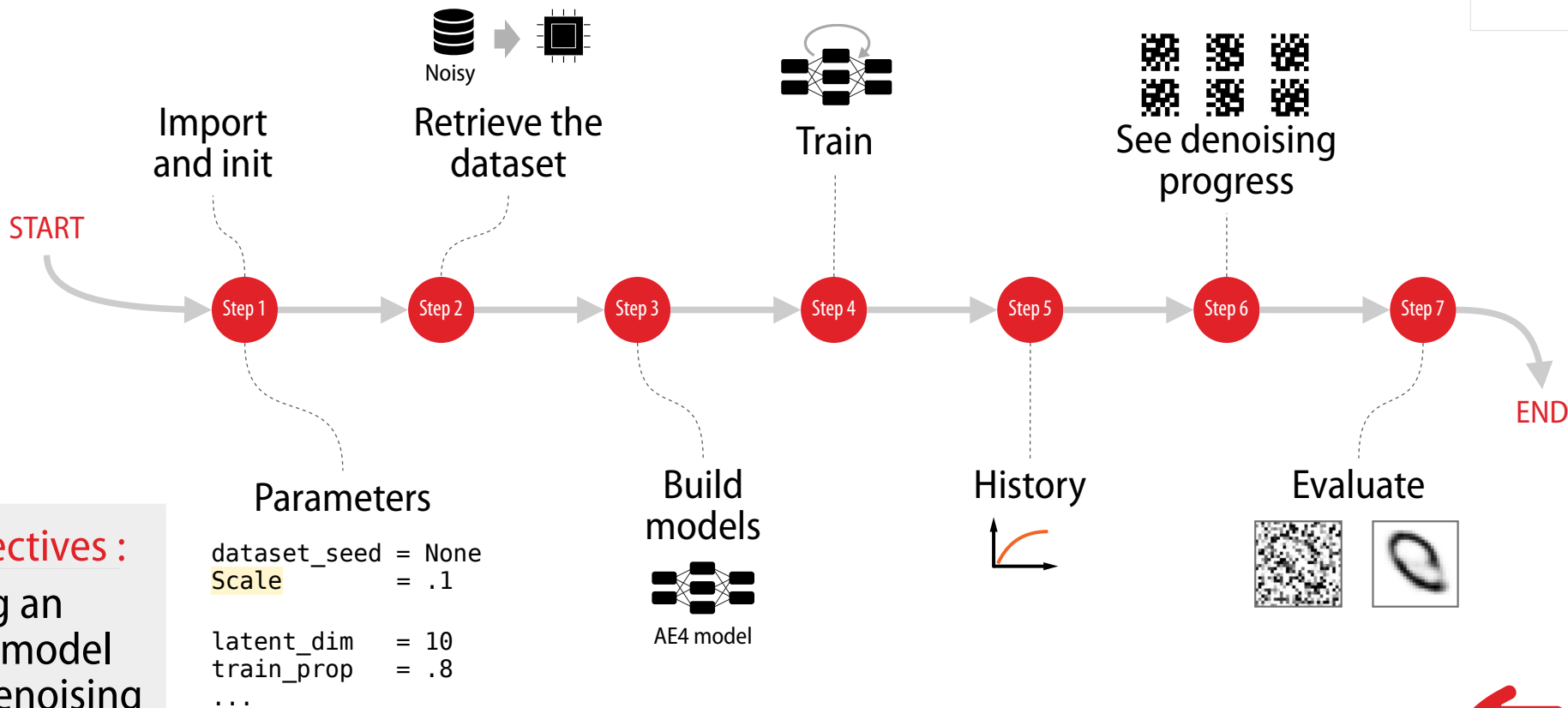


Objectives :

Using an
uniq model
for denoising
and classifing



20' on a CPU
With scale=1



Objectives :

Using an
uniq model
for denoising
and classifing



10



**Autoencoder
networks**

AE

8.1

Autoencoder network

→ Concepts and architecture

8.2

Keras

→ Functional API

8.3

Example 1 : AE1/3

→ Simple denoiser model



8.4

Example 2 : AE4

→ Dual output model : denoiser and classifier



8.5

Example 3 : AE5

→ Dual output model : denoiser and classifier
with inception approach





Adding Inception approach

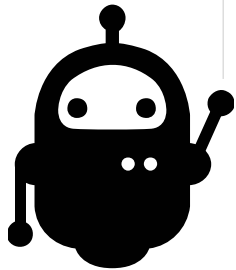
Notebook : [\[AE5\]](#)

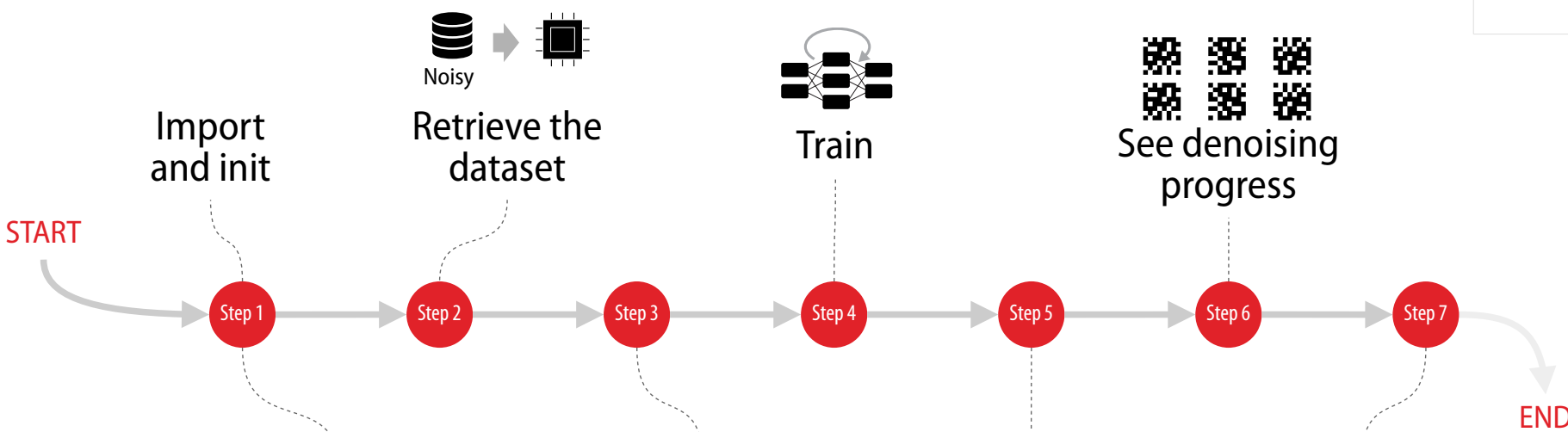
Objective :

Adding an inception part to our previous model
Understand the Keras **functional API**

Dataset :

MNIST





Import and init

Retrieve the dataset

Train

See denoising progress

Parameters

Build models

History

Evaluate

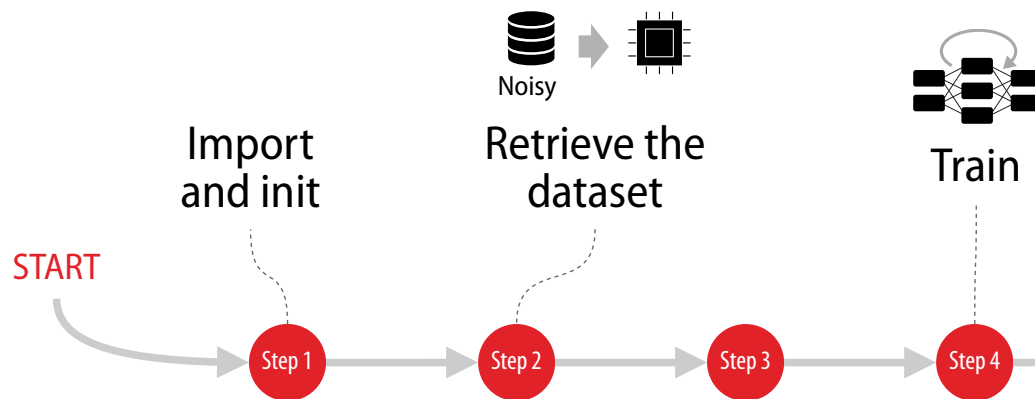
```
dataset_seed = None
Scale        = .1
latent_dim   = 10
train_prop   = .8
...
```

AE5 model



Objectives :
Using an
uniq model
for denoising
and classifing





Parameters

```

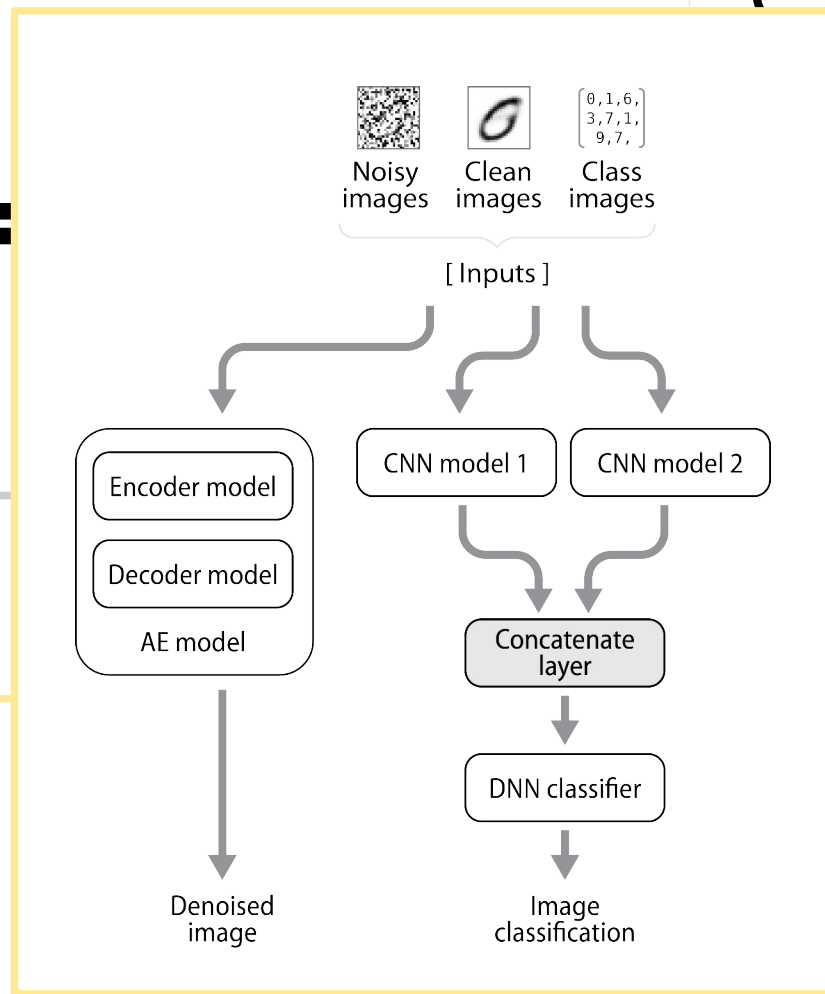
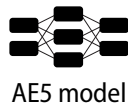
dataset_seed = None
Scale        = .1
latent_dim   = 10
train_prop   = .8
...

```

Objectives :

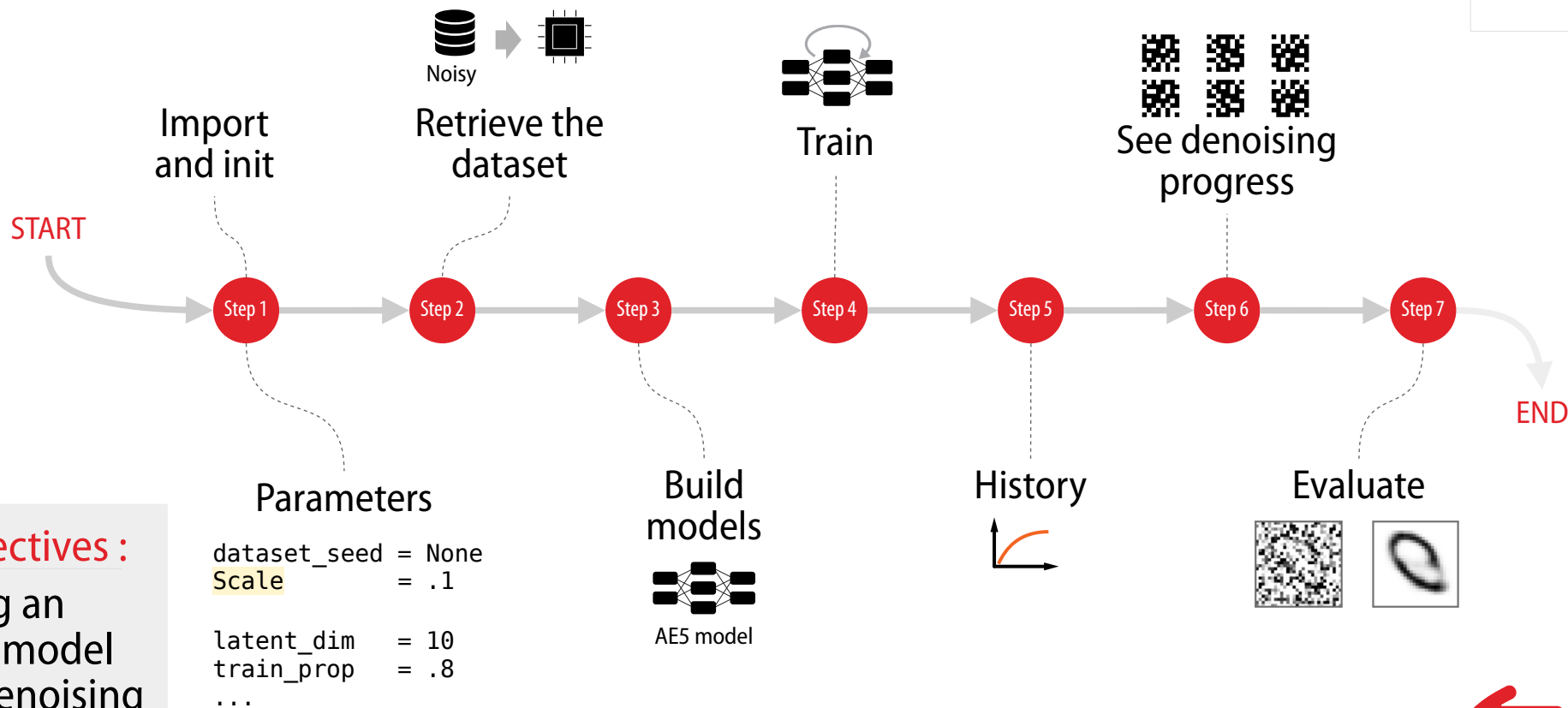
Using an
uniq model
for denoising
and classifing

Build models





20' on a CPU
With scale=1



Objectives :

Using an
uniq model
for denoising
and classifing





Little things and concepts to **keep in mind**

- Unsupervised learning allows many things
- The world of convolutions is vast
- Don't forget 1D convolutions
- Keras functional API is good for you !
- Multi-input models
- Multi-output models
- Inception models
- GPUS is also good
- Think about structuring the codes (python modules)
- (and to document them!)

Next, on Fidle :

11



**Variational
Autoencoder**

VAE



Jeudi 9 février,

Épisode 11 :

Variational Autoencoder (VAE) "self supervised" and generative networks

Principes et architecture d'un Variational AutoEncodeur (VAE)

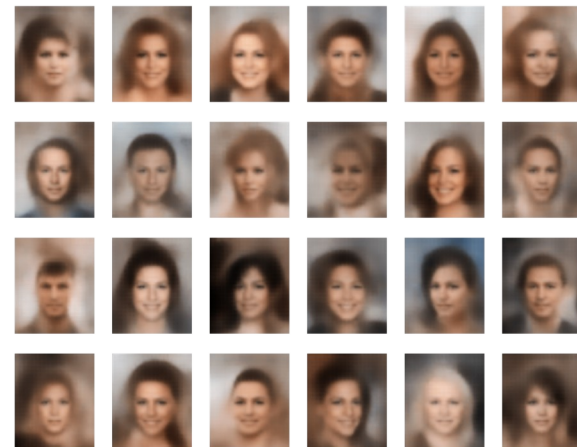
L'espace latent : the place to be !

Projection gaussienne - Génération de données - Morphing dans l'espace latent

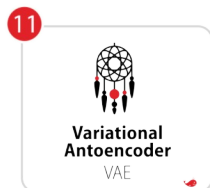
Datasets clusterisés - GPU et batch

Exemple proposé : Génération de portraits

Durée : 2h

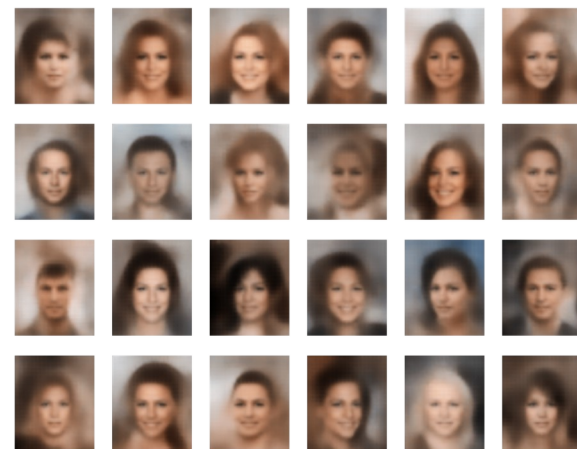
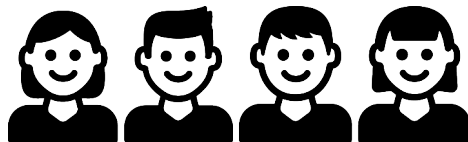


Next on Fidle :



Jeudi 9 février,
Séquence 11 :
Variational Autoencoder (VAE)
"self supervised" and generative networks

Merci !



Exemple de génération de données
via un espace latent

To be continued...